一、实验题目

现有数据集 MLP_data.csv 包含了以下特征：

longitude： 经度

latitude： 纬度

housing_age： 房龄

homeowner_income： 房主收入，单位为“十万元”

house_price： 房价

本次实验忽略特征 longitude 以及 latitude ，训练感知机 `f(housing_age, homeowner_income )`

拟合数据 `(housing_age, homeowner_income, house_price )`

二、实验内容

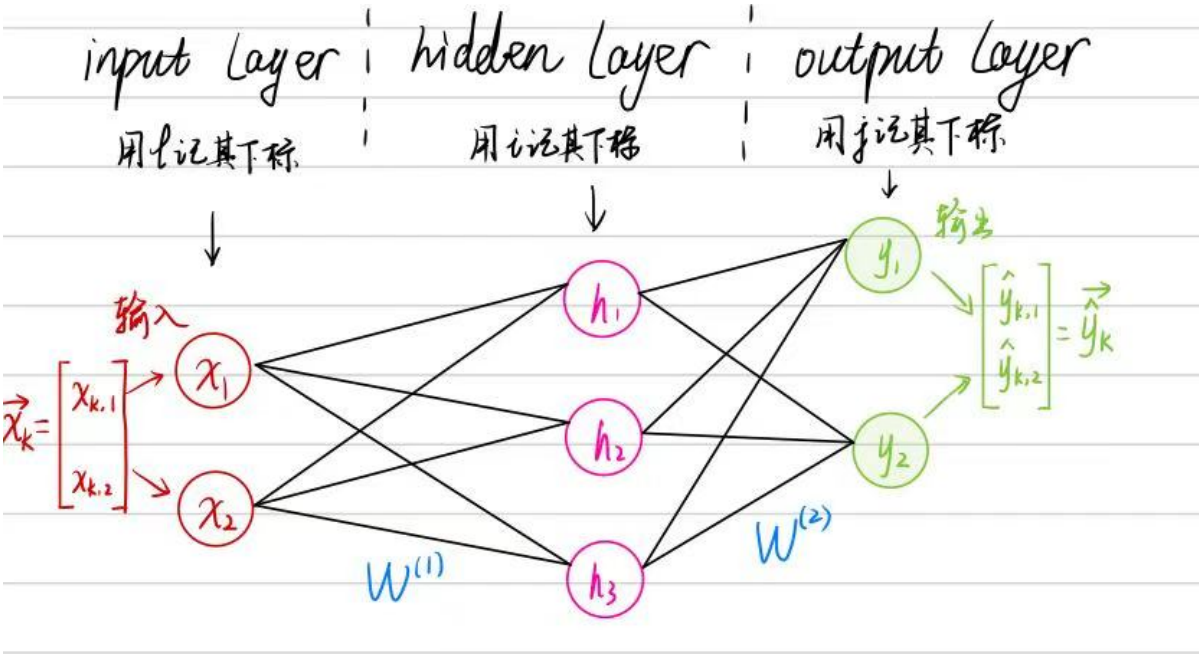
1.算法原理

不失一般性，假设输入层有2个结点，隐藏层有3个结点，输出层有2个结点

假设样本集合为D，共有m个样本

$$D=\{(\mathbf{x_k},\mathbf{y_k})\}_{k=1}^{m}$$

那么，多层感知机MLP可以表示为



1.1前向传播

记激活函数为 $\mathbf{g}$

假如没有偏置项

$$\mathbf{h_k} = g(W^{(1)} \cdot \mathbf{x_k})$$
$$\mathbf{y_k} = g(W^{(2)} \cdot \mathbf{h_k})$$

从而

$$\mathbf{y_k} = g(W^{(2)} \cdot g(W^{(1)} \cdot \mathbf{x_k}))$$

假如有偏置项

$$\mathbf{h_k} = g(W^{(1)} \cdot \mathbf{x_k} + \mathbf{b}^{(1)})$$
$$\mathbf{y_k} = g(W^{(2)} \cdot \mathbf{h_k} + \mathbf{b}^{(2)})$$

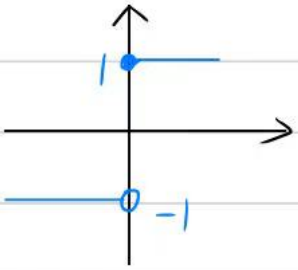
从而

$$\mathbf{y_k} = g(W^{(2)} \cdot g(W^{(1)} \cdot \mathbf{x_k} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)})$$

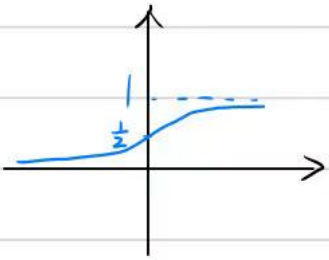
1.2 激活函数

常见的激活函数有

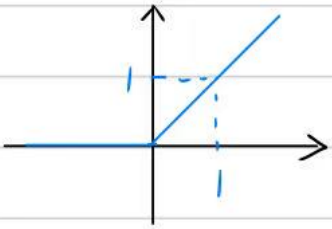
$$sgn(x) = \begin{cases} 1, & x > 0 \\ -1, & x < 0 \end{cases}$$



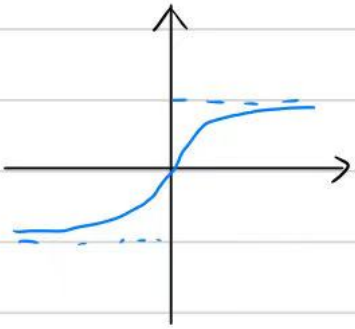
$$sigmoid(x) = \frac{1}{1+e^{-x}}$$



$$ReLU(x) = max(0, x)$$



$$tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



1.3 损失函数

使用均方误差

$$E_k = \frac{1}{2} \sum_j (\hat{y}_{j,k} - y_{j,k})^2$$

(这里的1/2是网络后面的求导方便)

1.4 后向传播

利用梯度下降法进行反向传播，具体计算如下：

反向传播到第2层

$$\Delta b_j^{(2)} = -\eta \frac{\partial E}{\partial y_{in,j}} \qquad \Delta w_{ij}^{(2)} = -\eta \frac{\partial E}{\partial w_{ij}}$$

由链式法则 $\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial y_{in,j}} \cdot \frac{\partial y_{in,j}}{\partial w_{ij}^{(2)}}$

$\because E = \frac{1}{2}\sum_{j=1}^{n}(\hat{y}_j - y_j)^2$

对于sigmod函数 $f(x) = \frac{1}{1+e^{-x}}$

$f'(x) = f(x)[1-f(x)]$

$\because y_{in,j} = \sum_{i=1}^{m} w_{ij}^{(2)} h_i$

$\therefore \frac{\partial E}{\partial y_j} = \hat{y}_j - y_j \qquad \therefore \frac{\partial \hat{y}_j}{\partial y_{in,j}} = \hat{y}_j(1-\hat{y}_j) \qquad \therefore \frac{\partial y_{in,j}}{\partial w_{ij}^{(2)}} = h_i$

$$\frac{\partial E}{y_{in,j}} = \frac{\partial E}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial y_{in,j}} = (\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j) \qquad (1)$$

$\therefore \Delta b_j^{(2)} = -\eta \underline{(\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j)} \rightarrow$ 记为 $g_j^{(2)}$

$\therefore \Delta w_{ij}^{(2)} = -\eta \underline{(\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j)} h_i$

第2层

更新 $\begin{cases} b_j^{(2)} \leftarrow b_j^{(2)} + \Delta b_j^{(2)} \\ w_{ij}^{(2)} \leftarrow w_{ij}^{(2)} + \Delta w_{ij}^{(2)} \end{cases}$

反向传播到第1层

$$\Delta b_i^{(1)} = -\eta \frac{\partial E}{\partial h_{in,i}} \qquad \Delta w_{li}^{(1)} = -\eta \frac{\partial E}{\partial w_{li}^{(1)}}$$

由链式法则 $\frac{\partial E}{\partial w_{li}^{(1)}} = \frac{\partial E}{\partial \hat{h}_i} \frac{\partial \hat{h}_i}{\partial h_{in,i}} \frac{\partial h_{in,i}}{\partial w_{li}^{(1)}}$

$$\frac{\partial E}{\partial \hat{h}_i} = \sum_j \frac{\partial E}{\partial y_{in,j}} \frac{\partial y_{in,j}}{\partial \hat{h}_i}$$

对于sigmod函数 $f(x) = \frac{1}{1+e^{-x}}$

$$f'(x) = f(x)[1-f(x)]$$

$$\therefore \frac{\partial \hat{h}_i}{\partial h_{in,i}} = \hat{h}_i(1-\hat{h}_i)$$

$$\because h_{in,i} = \sum_l w_{li}^{(1)} x_l + b^{(1)}$$

$$\therefore \frac{\partial h_{in,i}}{\partial w_{li}^{(1)}} = x_l$$

由(1)式

$$\frac{\partial E}{\partial y_{in,j}} = (\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j)$$

$$\because y_{in,j} = \sum_i w_{ij}^{(2)} \hat{h}_i$$

$$\therefore \frac{\partial y_{in,j}}{\partial \hat{h}_i} = w_{ij}^{(2)}$$

$$\therefore \Delta b_i^{(1)} = -\eta \sum_j (\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j) w_{ij}^{(2)} \hat{h}_i(1-\hat{h}_i)$$

$$= -\eta \underbrace{\hat{h}_i(1-\hat{h}_i) \sum_j (\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j) w_{ij}^{(2)}}_{g_i^{(1)}}$$

$$\Delta w_{li}^{(1)} = -\eta \sum_j (\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j) w_{ij}^{(2)} \hat{h}_i(1-\hat{h}_i) x_l$$

$$= -\eta x_l \underbrace{\hat{h}_i(1-\hat{h}_i) \sum_j (\hat{y}_j - y_j)\hat{y}_j(1-\hat{y}_j) w_{ij}^{(2)}}_{g_i^{(1)}}$$

第1层

更新 $\begin{cases} b_i^{(1)} \leftarrow b_i^{(1)} + \Delta b_i^{(1)} \\ w_{li}^{(1)} \leftarrow w_{li}^{(1)} + \Delta w_{li}^{(1)} \end{cases}$

1.5 算法流程

$$\{(\vec{x}_k, y_k)\}_{k=1}^{n}$$

初始化权重和偏置项

repeat

for $(\vec{x}_k, \vec{y}_k) \in D$:

// 前向传播

$$\hat{\vec{y}}_k = g\left(W^{(2)} g\left(W^{(1)} \vec{x}_k + b^{(1)}\right) + b^{(2)}\right)$$

// 计算误差

$$E_k = \frac{1}{2}\left(\hat{\vec{y}}_k - \vec{y}_k\right)^T \left(\hat{y}_k - y_k\right)$$

// 反向传播

求 $\Delta b^{(2)}$, $\Delta W^{(2)}$

求 $\Delta b^{(1)}$, $\Delta W^{(1)}$

// 更新

$b^{(2)} \leftarrow b^{(2)} + \Delta b^{(2)}$, $W^{(2)} \leftarrow W^{(2)} + \Delta W^{(2)}$

$b^{(1)} \leftarrow b^{(1)} + \Delta b^{(1)}$ $W^{(1)} \leftarrow W^{(1)} + \Delta W^{(1)}$

end for

until 达到停止条件

2.关键代码展示

2.1 前向传播

经过实验发现，输出层的激活函数使用线性输出比sigmod函数的效果更好，所以最终采用了线性输出

```
def forward(self, X):
    # 前向传播
    self.h_in = X @ self.w1 + self.b1
    self.h_out = self.sigmod(self.h_in)
    self.y_in = self.h_out @ self.w2 + self.b2
    self.y_out = self.y_in  # 线性输出
    return self.y_out
```

2.2 反向传播

在参数更新中，我采用动量梯度下降算法。

这样做的一个直观表现就是，如果目标函数最近 n 个时间步的梯度方向比较一致，那么 t 时刻的梯度加上t-1时刻的速度会让 t 时刻的速度变量较大，则模型参数的改变量就大；若这几个时间步内梯度方向变化较大，会造成其加权平均值较小，则模型参数的改变量就相对小。

从而有利于更快接近最优解，避免陷入局部最优解。

```
def backward(self,X,y):
    m=X.shape[0]

    # 输出层梯度（线性激活）
    dL_dy_out = (self.y_out - y)
    dW2=(self.h_out.T@dL_dy_out)/m
    db2=np.sum(dL_dy_out,axis=0)/m

    # 隐藏层梯度
    dL_dh_out = dL_dy_out @ self.w2.T
    dL_dh_in = dL_dh_out * self.h_out * (1 - self.h_out)
    dW1 = (X.T @ dL_dh_in) / m
    db1 = np.sum(dL_dh_in, axis=0) / m

    # 动量式更新
    # 动量更新
    self.momentum_w2 = self.beta * self.momentum_w2 + (1 - self.beta) * dW2
    self.momentum_b2 = self.beta * self.momentum_b2 + (1 - self.beta) * db2
    self.momentum_w1 = self.beta * self.momentum_w1 + (1 - self.beta) * dW1
    self.momentum_b1 = self.beta * self.momentum_b1 + (1 - self.beta) * db1

    # 参数更新
    self.w2 -= self.learning_rate * self.momentum_w2
    self.b2 -= self.learning_rate * self.momentum_b2
    self.w1 -= self.learning_rate * self.momentum_w1
    self.b1 -= self.learning_rate * self.momentum_b1
```

2.3 训练过程主函数

在训练过程中，我采用了批训练的方式。一方面，相比于逐个样本进行训练，批训练有利于减少迭代次数，加快训练速度；另一方面，相比于全部样本一起训练，批训练有利于减少内存占用。

在每轮训练后，我通过计算验证集的均方根误差，进行早停判断，能够**避免过拟合**的问题。

训练的主要过程如下：

```
def train(self,X_train,y_train,X_validate,y_validate,epochs=100,
batch_size=32,patience_rate=0.5):

        all_train_loss=[]
        all_validate_loss=[]
        best_loss = np.inf
        no_improve = 0
        patience_size=patience_rate*epochs

        for epoch in range(epochs):
            # 训练
            permutation = np.random.permutation(len(X_train))
            X_shuffled = X_train[permutation]
            y_shuffled = y_train[permutation]
            for i in range(0, len(X_train), batch_size):
                batch_indices = permutation[i:i + batch_size]
                X_batch = X_shuffled[batch_indices]
                y_batch = y_shuffled[batch_indices]

                self.forward(X_batch)
                self.backward(X_batch,y_batch)

            y_train_pred=self.forward(X_train)
            train_loss=self.loss(y_train_pred,y_train)
            all_train_loss.append(train_loss)

            y_validate_pred=self.forward(X_validate)
            validate_loss=self.loss(y_validate_pred,y_validate)
            all_validate_loss.append(validate_loss)

            # 早停判断
            if validate_loss < best_loss:
                best_loss = validate_loss
                no_improve = 0
                # 保存最佳参数
                self.best_params['w1']=self.w1.copy()
                self.best_params['b1']= self.b1.copy()
                self.best_params['w2']= self.w2.copy()
                self.best_params['b2']=self.b2.copy()

            else:
                no_improve += 1
                if no_improve >= patience_size:
                    print(f"Early stopping at epoch {epoch}")

                    self.w1 = self.best_params['w1']
                    self.b1 = self.best_params['b1']
```

```
                    self.w2 = self.best_params['w2']
                    self.b2 = self.best_params['b2']
                    break

            print(f"Epoch {epoch + 1}/{epochs} | Train Loss: {train_loss:.4f} |
Val Loss: {validate_loss:.4f}")

        return all_train_loss,all_validate_loss
```

3.创新点

3.1 动量梯度下降算法

这样做的一个直观表现就是，如果目标函数最近 n 个时间步的梯度方向比较一致，那么 t 时刻的梯度加上t-1时刻的速度会让 t 时刻的速度变量较大，则模型参数的改变量就大；若这几个时间步内梯度方向变化较大，会造成其加权平均值较小，则模型参数的改变量就相对小。

从而有利于更快接近最优解，避免陷入局部最优解。

对应代码如下：

```
# 动量更新
self.momentum_w2 = self.beta * self.momentum_w2 + (1 - self.beta) * dw2
self.momentum_b2 = self.beta * self.momentum_b2 + (1 - self.beta) * db2
self.momentum_w1 = self.beta * self.momentum_w1 + (1 - self.beta) * dw1
self.momentum_b1 = self.beta * self.momentum_b1 + (1 - self.beta) * db1

# 参数更新
self.w2 -= self.learning_rate * self.momentum_w2
self.b2 -= self.learning_rate * self.momentum_b2
self.w1 -= self.learning_rate * self.momentum_w1
self.b1 -= self.learning_rate * self.momentum_b1
```

3.2 批处理

我采用批训练的方式。一方面，相比于逐个样本进行训练，批训练有利于减少迭代次数，加快训练速度；另一方面，相比于全部样本一起训练，批训练有利于减少内存占用。

对应代码如下：

```
# 训练
permutation = np.random.permutation(len(X_train))
X_shuffled = X_train[permutation]
y_shuffled = y_train[permutation]
for i in range(0, len(X_train), batch_size):
    batch_indices = permutation[i:i + batch_size]
    X_batch = X_shuffled[batch_indices]
    y_batch = y_shuffled[batch_indices]

    self.forward(X_batch)
    self.backward(X_batch,y_batch)
```

3.3 早停判断

计算验证集的均方根误差，进行早停判断，能够**避免过拟合**的问题。

对应代码如下：

```
# 早停判断
if validate_loss < best_loss:
    best_loss = validate_loss
    no_improve = 0
    # 保存最佳参数
    self.best_params['w1']=self.w1.copy()
    self.best_params['b1']= self.b1.copy()
    self.best_params['w2']= self.w2.copy()
    self.best_params['b2']=self.b2.copy()

else:
    no_improve += 1
    if no_improve >= patience_size:
        print(f"Early stopping at epoch {epoch}")

        self.w1 = self.best_params['w1']
        self.b1 = self.best_params['b1']
        self.w2 = self.best_params['w2']
        self.b2 = self.best_params['b2']
        break
```

4.完整代码

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import cm

# 数据预处理
df = pd.read_csv("MLP_data.csv")
df_choose = df.iloc[:, 2:]

X_data = df_choose.iloc[:, :2]
y_data = df_choose.iloc[:, 2:]

# 标准化
X_mean, X_std = np.mean(X_data, axis=0), np.std(X_data, axis=0)
y_mean, y_std = np.mean(y_data, axis=0), np.std(y_data, axis=0)

X_processed = (X_data - X_mean) / X_std
y_processed = (y_data - y_mean) / y_std

X = np.array(X_processed)
y = np.array(y_processed)
```

```python
# 划分训练集、验证集和测试集
indices = np.random.permutation(len(X))
train_size = int(0.7 * len(X))
validate_size=int(0.15 * len(X))

X_train, y_train = X[indices[:train_size]], y[indices[:train_size]]
X_validate,y_validate=X[indices[train_size:train_size+validate_size]],y[indices[t
rain_size:train_size+validate_size]]
X_test, y_test = X[indices[train_size+validate_size:]],
y[indices[train_size+validate_size:]]

np.random.seed(10)

# 网络参数
input_size = 2
hidden_size = 16
output_size = 1

class MLP:
    def __init__(self,learning_rate):
        self.learning_rate=learning_rate
        self.b1 = np.zeros(hidden_size)
        self.w1 = np.random.rand(input_size, hidden_size)
        self.b2 = np.zeros(output_size)
        self.w2 = np.random.rand(hidden_size, output_size)
        self.best_params={}

        # 动量优化
        self.momentum_w1 = np.zeros_like(self.w1)
        self.momentum_b1 = np.zeros_like(self.b1)
        self.momentum_w2 = np.zeros_like(self.w2)
        self.momentum_b2 = np.zeros_like(self.b2)
        self.beta = 0.9

    def sigmod(self,x):
        return 1 / (1 + np.exp(-x))

    def forward(self, x):
        # 前向传播
        self.h_in = x @ self.w1 + self.b1
        self.h_out = self.sigmod(self.h_in)
        self.y_in = self.h_out @ self.w2 + self.b2
        self.y_out = self.y_in  # 线性输出

        return self.y_out

    def loss(self, y_pred, y):
        return 0.5 * np.mean((y_pred - y) ** 2)

    def backward(self,x,y):
        m=x.shape[0]

        # 输出层梯度（线性激活）
        dL_dy_out = (self.y_out - y)
        dw2=(self.h_out.T@dL_dy_out)/m
        db2=np.sum(dL_dy_out,axis=0)/m
```

```
        # 隐藏层梯度
        dL_dh_out = dL_dy_out @ self.w2.T
        dL_dh_in = dL_dh_out * self.h_out * (1 - self.h_out)
        dw1 = (X.T @ dL_dh_in) / m
        db1 = np.sum(dL_dh_in, axis=0) / m

        # 动量式更新
        # 动量更新
        self.momentum_w2 = self.beta * self.momentum_w2 + (1 - self.beta) * dw2
        self.momentum_b2 = self.beta * self.momentum_b2 + (1 - self.beta) * db2
        self.momentum_w1 = self.beta * self.momentum_w1 + (1 - self.beta) * dw1
        self.momentum_b1 = self.beta * self.momentum_b1 + (1 - self.beta) * db1

        # 参数更新
        self.w2 -= self.learning_rate * self.momentum_w2
        self.b2 -= self.learning_rate * self.momentum_b2
        self.w1 -= self.learning_rate * self.momentum_w1
        self.b1 -= self.learning_rate * self.momentum_b1

    def train(self,X_train,y_train,X_validate,y_validate,epochs=100,
batch_size=32,patience_rate=0.5):

        all_train_loss=[]
        all_validate_loss=[]
        best_loss = np.inf
        no_improve = 0
        patience_size=patience_rate*epochs

        for epoch in range(epochs):
            # 训练
            permutation = np.random.permutation(len(X_train))
            X_shuffled = X_train[permutation]
            y_shuffled = y_train[permutation]
            for i in range(0, len(X_train), batch_size):
                batch_indices = permutation[i:i + batch_size]
                X_batch = X_shuffled[batch_indices]
                y_batch = y_shuffled[batch_indices]

                self.forward(X_batch)
                self.backward(X_batch,y_batch)

            y_train_pred=self.forward(X_train)
            train_loss=self.loss(y_train_pred,y_train)
            all_train_loss.append(train_loss)

            y_validate_pred=self.forward(X_validate)
            validate_loss=self.loss(y_validate_pred,y_validate)
            all_validate_loss.append(validate_loss)

            # 早停判断
            if validate_loss < best_loss:
                best_loss = validate_loss
                no_improve = 0
                # 保存最佳参数
                self.best_params['w1']=self.w1.copy()
```

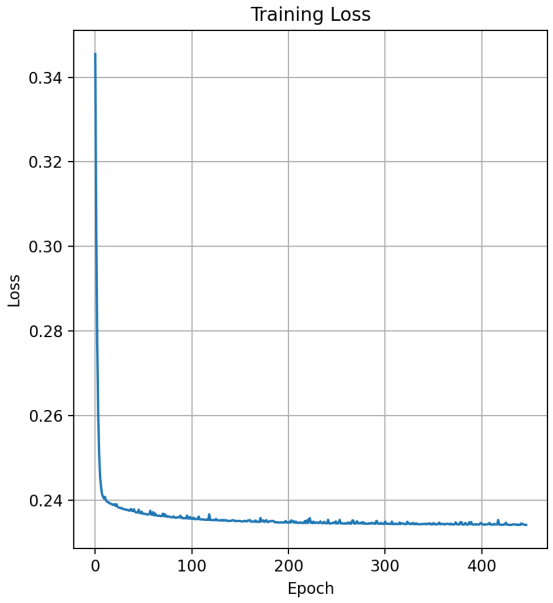
```
                self.best_params['b1']= self.b1.copy()
                self.best_params['W2']= self.w2.copy()
                self.best_params['b2']=self.b2.copy()

            else:
                no_improve += 1
                if no_improve >= patience_size:
                    print(f"Early stopping at epoch {epoch}")

                    self.w1 = self.best_params['w1']
                    self.b1 = self.best_params['b1']
                    self.w2 = self.best_params['w2']
                    self.b2 = self.best_params['b2']
                    break

            print(f"Epoch {epoch + 1}/{epochs} | Train Loss: {train_loss:.4f} |
Val Loss: {validate_loss:.4f}")

        return all_train_loss,all_validate_loss

    def predict(self,X):
        return self.forward(X)

def plot_3d_surface(ax, model, X):
    # 生成网格数据
    x_min, x_max = X[:, 0].min(), X[:, 0].max()
    y_min, y_max = X[:, 1].min(), X[:, 1].max()
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 50),
                         np.linspace(y_min, y_max, 50))

    # 预测网格点
    grid = np.c_[xx.ravel(), yy.ravel()]
    zz = model.predict(grid).reshape(xx.shape)

    # 绘制曲面
    surf = ax.plot_surface(xx, yy, zz, cmap=cm.coolwarm,
                           alpha=0.6, linewidth=0, antialiased=True)
    return surf


if __name__ == "__main__":
    # 初始化模型
    mlp = MLP(learning_rate=0.01)

    # 训练模型
    all_train_loss,all_validate_loss = mlp.train(X_train,
y_train,X_validate,y_validate,
                                                 epochs=1000,
                                                 batch_size=32,
                                                 patience_rate=0.1
                                                 )

    y_pred=mlp.predict(X_test)
    y_pred=y_pred*np.array(y_std)+np.array(y_mean)

    y_test_real=y_test*np.array(y_std)+np.array(y_mean)
```

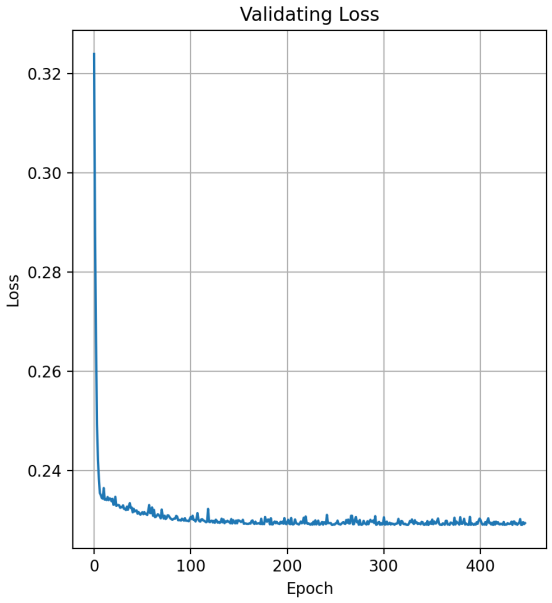
```
predict_loss=np.sqrt(np.average((y_pred-y_test_real)**2))

print("y_pred:\n",y_pred)
print("predict_loss:\n",predict_loss)

# 绘制损失曲线
fig=plt.figure(figsize=(12, 6))
ax1=fig.add_subplot(121)
ax1.plot(all_train_loss)
ax1.set_title("Training Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")
plt.grid(True)

ax2 = fig.add_subplot(122)
ax2.plot(all_validate_loss)
ax2.set_title("Validating Loss")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Loss")
plt.grid(True)
plt.show()

fig = plt.figure(figsize=(12, 6))

# 训练集子图
ax1 = fig.add_subplot(121, projection='3d')
ax1.scatter(X_train[:, 0], X_train[:, 1], y_train, c='b', label='True', s=10)
plot_3d_surface(ax1, mlp, X_train)
ax1.set_xlabel("Housing Age (scaled)")
ax1.set_ylabel("Income (scaled)")
ax1.set_zlabel("Price (scaled)")
ax1.set_title("Training Set")

# 测试集子图
ax2 = fig.add_subplot(122, projection='3d')
ax2.scatter(X_test[:, 0], X_test[:, 1], y_test, c='r', label='True', s=10)
plot_3d_surface(ax2, mlp, X_test)
ax2.set_xlabel("Housing Age (scaled)")
ax2.set_ylabel("Income (scaled)")
ax2.set_zlabel("Price (scaled)")
ax2.set_title("Test Set")

plt.tight_layout()
plt.show()
```

三、实验结果及分析

1.损失曲线

训练集和验证集的损失曲线分别如下：





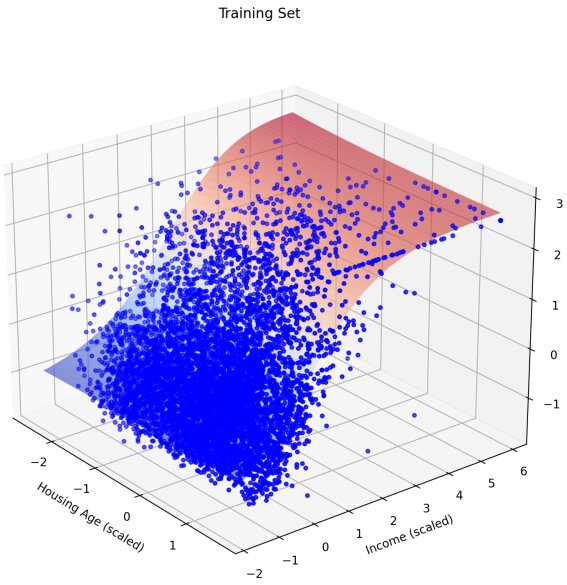
由图可见，训练集和验证集的收敛速度都比较快，说明模型已经收敛。而且，训练次数少于指定的1000次，说明早停判断起了作用，能够有利于避免过拟合。

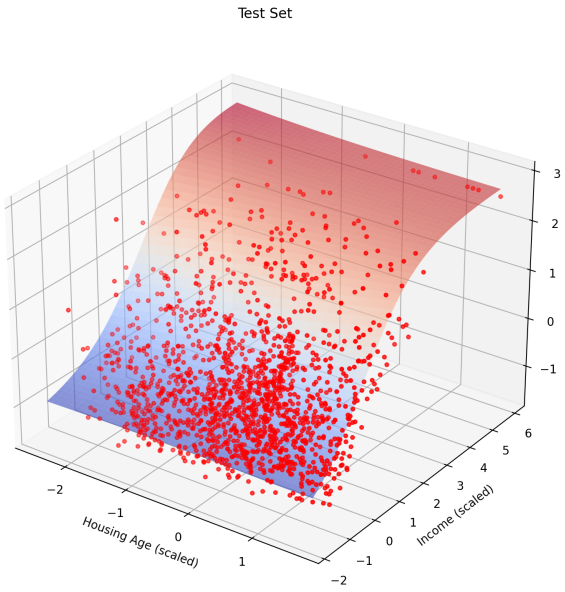
2.测试集测试结果

代入测试集数据

最终预测结果的RMSE= 74741.30762238539

训练集和测试集拟合效果的三维图分别如下：





由三维图和测试集的RMSE可见，对于大部分的点，模型的拟合效果较好，但是由于少部分点偏离正常范围（噪声点），导致最终的RMSE较大。这可能是数据集本身的问题所导致的。

若要进一步提高拟合效果，一方面，可以增加隐藏层的层数，另一方面，可以先剔除噪声点。此外，数据集中还可以增加自变量的维度，比如：提供房子的面积、房子的装修费用等，有利于提高模型拟合效果。

四、参考资料

[1] artificial intelligence - What's is the difference between train, validation and test set, in neural networks? - Stack Overflow

[2] 理解梯度下降算法中的动量 - 知乎